

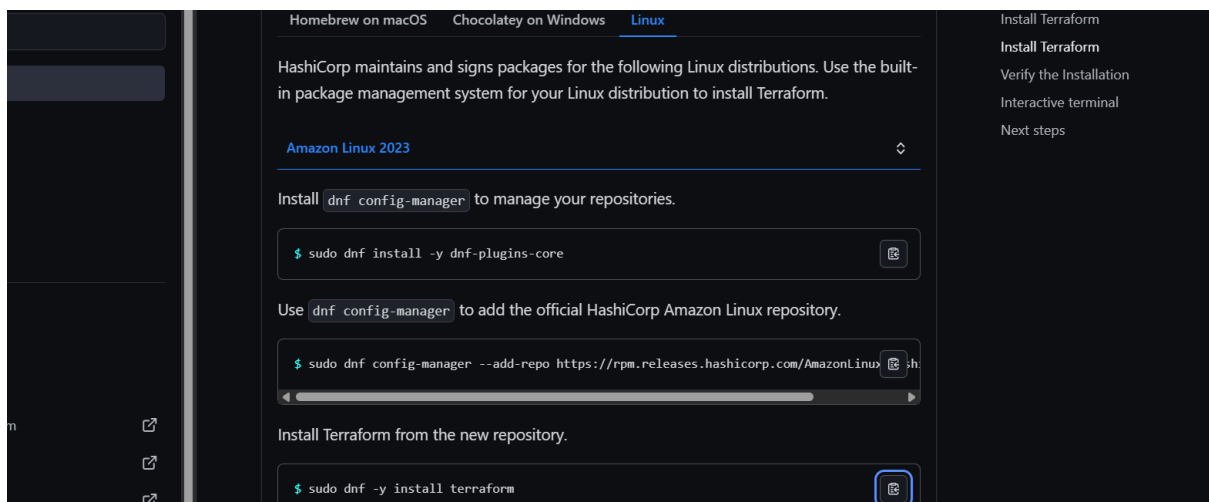
To begin with the Lab

Installation and Setup for Terraform with Jenkins

Summary of the Lab

In this lab, Terraform is installed on an Amazon Linux EC2 instance using DNF commands and configured with Jenkins by installing the Terraform plugin and setting its local path. The EC2 instance is assigned an IAM role with Administrator Access to ensure proper permissions. A new Jenkins pipeline job named terraformcheck is created. It pulls a vpc_jenkinsfile script from a GitHub repository that automates infrastructure provisioning using Terraform. The pipeline executes Terraform commands (init, plan, apply), pauses for manual approval, and then runs destroy to clean up resources. It successfully creates and deletes a VPC named demo-vpc-terraform.

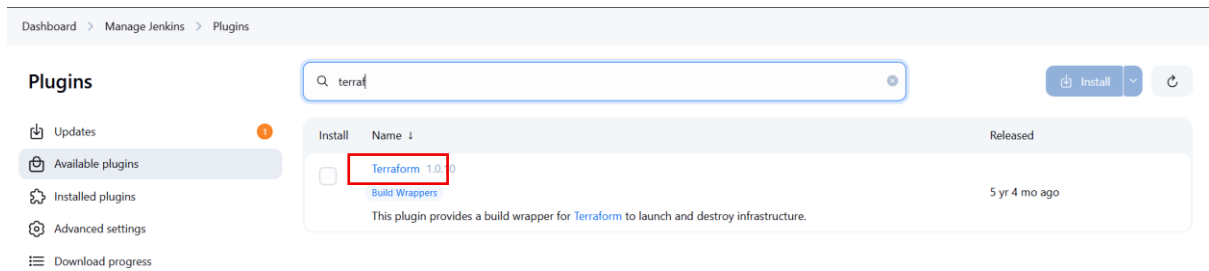
- Now we will install the terraform in our Amazon Linux on ec2 instance.
- You can also refer the documentation. Click [Here](#).
- Use the given below commands to install the terraform:
 - `sudo dnf install -y dnf-plugins-core`
 - `sudo dnf config-manager --add-repo`
<https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo>
 - `sudo dnf -y install terraform`



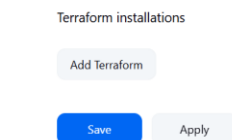
- Use the command to 'which terraform', it will show us the path.

```
[ec2-user@ip-172-31-21-149 ~]$ which terraform
/usr/bin/terraform
[ec2-user@ip-172-31-21-149 ~]$
```

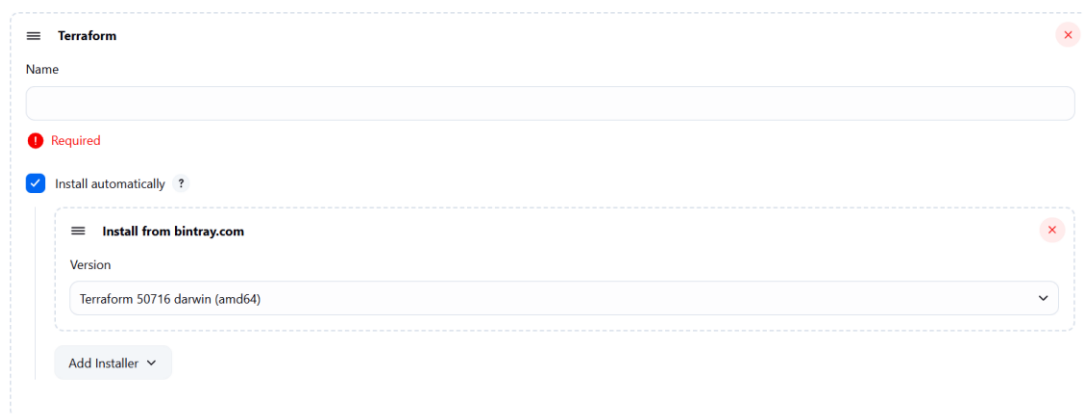
- Let's go back to Jenkins.
- Install the terraform plugin.



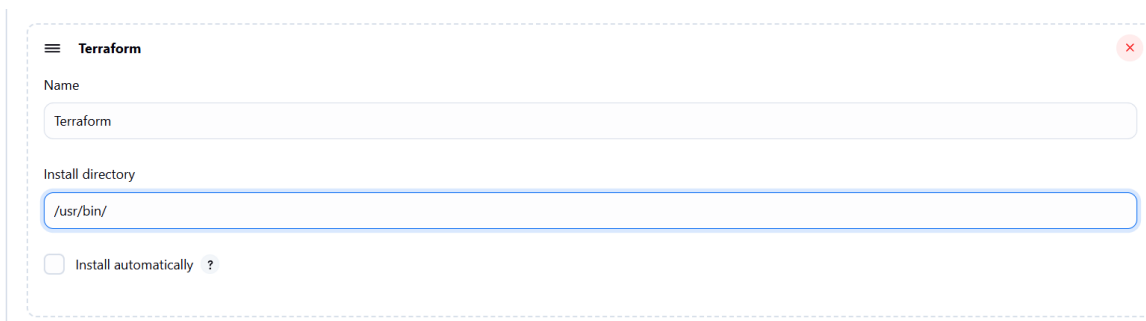
Now go to dashboard > Manage Jenkins > Tools > Terraform.



- Click Add Terraform.
- Uncheck install automatically.
- Delete this



- Now make the entries.
- Give the path of the terraform installed on the local machine.
- Click Save.



- Now there are lot of permission that you have to give to EC2 instance using IAM Role.
- So, there are lot of permission that we have to give so for now let's give the Administrator Access.
- Now we will create a new item.

- Click on the new item
- Give the name terraformcheck.
- Select the pipeline
- Click Ok
- In the pipeline section, select the hello world.
- And click Save
- Click build now.

Definition

Pipeline script

Script ?

```

1 pipeline {
2     agent any
3
4     stages {
5         stage('Hello') {
6             steps {
7                 echo 'Hello World'
8                 sh 'terraform version'
9             }
10        }
11    }
12 }
13

```

Use Groovy Sandbox ?

Dashboard > terraformcheck > #1

Status

Changes

Console Output

Edit Build Information

Delete build '#1'

Timings

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

Console Output

Download Copy View as plain text

Started by user Aman Rathor

[Pipeline] Start of Pipeline

[Pipeline] node

Running on Jenkins in /var/lib/jenkins/workspace/terraformcheck

[Pipeline] {

[Pipeline] stage

[Pipeline] { (Hello)

[Pipeline] echo

Hello World

[Pipeline] sh

+ terraform version

Terraform v1.12.2

on linux_amd64

[Pipeline] }

[Pipeline] // stage

[Pipeline] }

[Pipeline] // node

[Pipeline] End of Pipeline

Finished: SUCCESS

- Now go to the [repository](#)
- Click on the vpc_jenkinsfile.
- Copy the code the script code.

So, this Jenkins pipeline clones a GitHub repo containing Terraform code and provisions infrastructure using Terraform. It then pauses for manual approval before destroying the resources. The pipeline automates the full lifecycle: **init** → **plan** → **apply** → **optional destroy**.

- Now go to jenkins dashboard.
- Go to configure
- Paste the code in the pipeline.

Definition

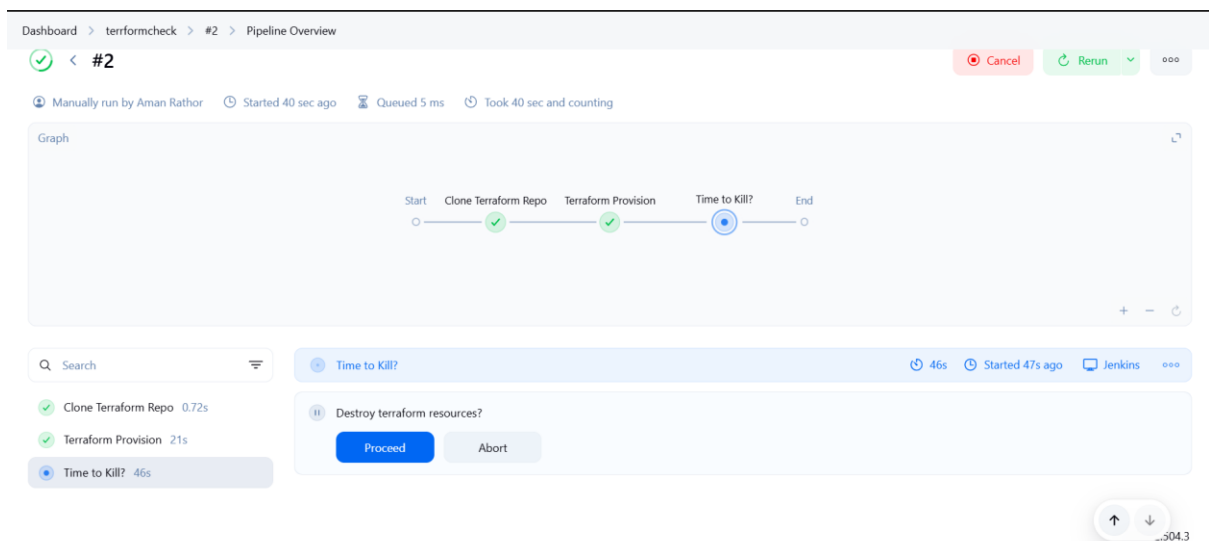
Pipeline script

```

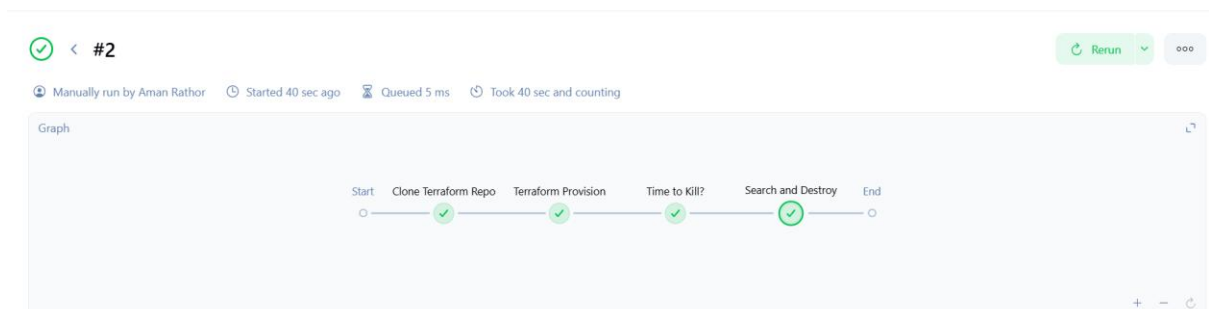
Script ?
stage('Clone Terraform Repo') {
    steps {
        echo 'Hello World'
        checkout([$class: 'GitSCM', branches: [[name: '*/main']], extensions: [], userRemoteConfigs: [[cr
    ]]
    }
}
stage('Terraform Provision') {
    steps {
        sh """
            cd terraform
            terraform init
            terraform plan
            terraform apply --auto-approve
        """
        sh 'pwd'
    }
}

```

- Click save and click build.
- Here you can see manual approval to destroy the resource



- Now I clicked on the proceed.
- Now it has destroyed the resource that it has created.



Q Search

✓ Clone Terraform Repo 0.72s

✓ Terraform Provision 21s

✓ Time to Kill? 1m 54s

✓ Search and Destroy 8.2s

cd terraform: terraform destroy --auto-approve 8.2s

```
41 - region = us-east-1 -> null
28 - tags = {
29   - "Name" = "demo-vpc-terraform"
30   - "Purpose" = "Jenkins Terraform Demo"
31 } -> null
32 - tags_all = {
33   - "Name" = "demo-vpc-terraform"
34   - "Purpose" = "Jenkins Terraform Demo"
35 } -> null
36 # (4 unchanged attributes hidden)
37 }
38
39 Plan: 0 to add, 0 to change, 1 to destroy.
40 aws_vpc.main: Destroying... [id=vpc-0d25a28ab6825cee9]
41 aws_vpc.main: Destruction complete after 1s
42
43 Destroy complete! Resources: 1 destroyed.
44
```

- This will create a new vpc with the name demo-vpc-terraform and it will also delete it.